

Data Engineering Ingestion

UA.DETI.ED - 2025/26

José Luís Oliveira

What is Data Ingestion?

❖ Defining Data Ingestion

- The process of extracting data from various source systems and moving it to a centralized storage environment.
- The critical first technical step in making raw data available for analytics and machine learning.



❖ Why It Matters

- Without reliable ingestion, downstream transformations, dashboards, and algorithms will fail.
- It establishes the baseline for data quality, latency, and governance across the organization.

The Data Engineering Lifecycle

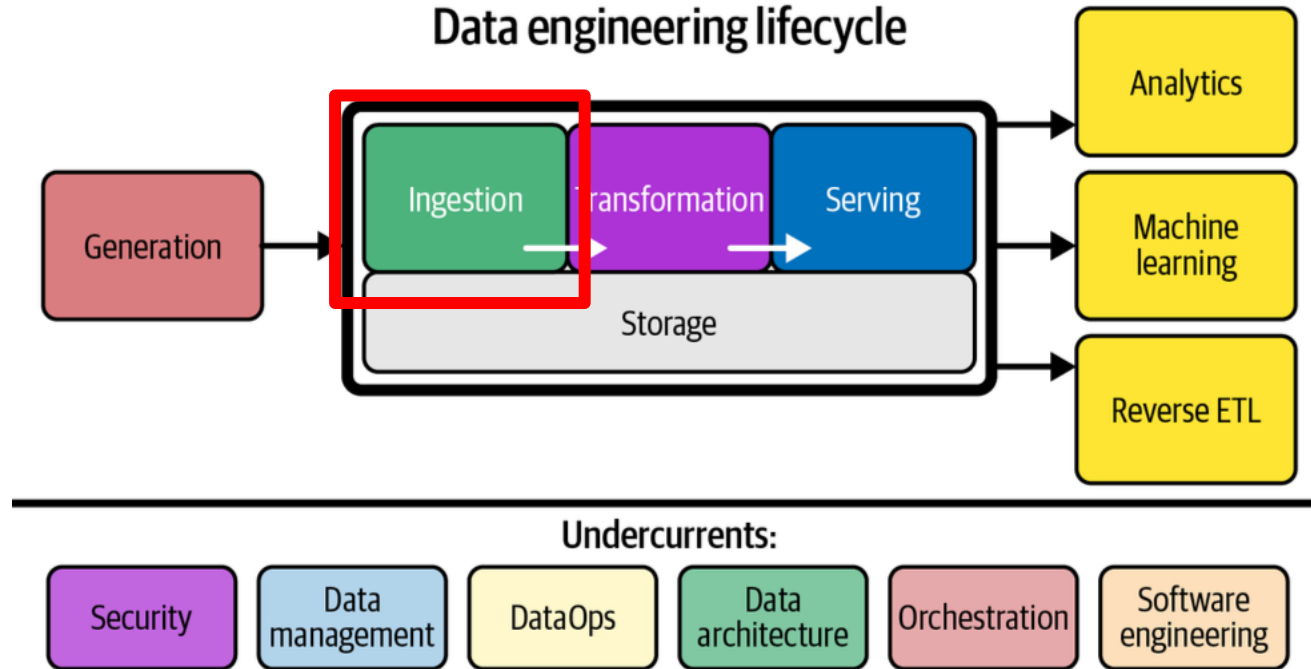
❖ Core Phases

- Generation: Where data is originally created by applications, IoT devices, or APIs.
- **Ingestion: The movement of data from the point of generation to the storage layer.**
- Storage: Landing the data in a reliable, accessible environment for future use.

❖ Downstream Activities

- Transformation: Cleaning, deduplicating, and structuring the raw data.
- Serving: Delivering refined data to BI tools and ML models.

The Data Engineering Lifecycle



Key Objectives of Data Ingestion

❖ Reliability and Robustness

- Ensuring data arrives safely, even during network failures or source system outages.
- Building fault-tolerant pipelines that gracefully recover from errors.

❖ Scalability and Performance

- Handling exponentially increasing data volumes without requiring complete pipeline rewrites.

❖ Accuracy and Completeness

- Guaranteeing no records are dropped or inadvertently duplicated during transit.

Engineering Considerations

- ❖ What is the primary use case or business value driving the need for this data?
- ❖ Where is this data ultimately landing ?
- ❖ How often should the data be updated from the source?
- ❖ What is the anticipated data volume today, and what is the projected growth trajectory?
- ❖ What format is the data in? Can downstream storage and transformation accept this format?
- ❖ Is the source data of good quality? What post-processing is required to serve it?
- ❖ Does the data require in-flight processing for downstream ingestion if the data is from a streaming source?

Source Systems: The Origin of Data

❖ Operational Databases

- Relational databases (MySQL, PostgreSQL) backing core business applications.
- NoSQL stores (MongoDB, Cassandra) handling document or key-value data.

❖ External APIs and SaaS Applications

- Pulling data from CRMs like Salesforce, marketing platforms, or external financial systems.

❖ Event Streams and Logs

- Capturing clickstream data from websites, IoT sensor telemetry, and server application logs.

Target Destinations

❖ Data Lakes

- Landing raw, unstructured, or semi-structured data cheaply and scalably (e.g., AWS S3).

❖ Data Warehouses

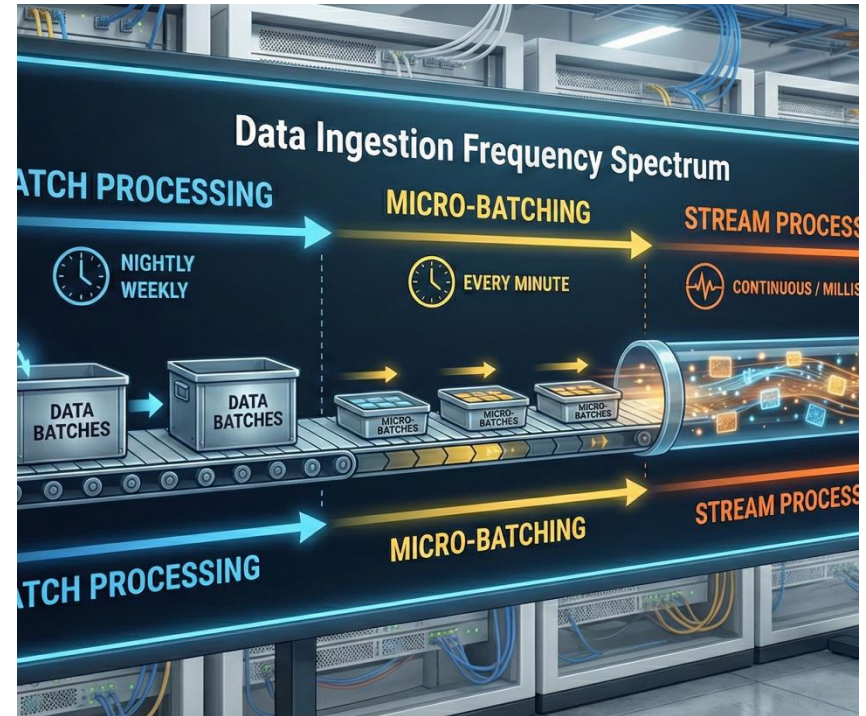
- Ingesting structured data directly for immediate analytical querying (e.g., Snowflake, Redshift).

❖ Data Lakehouses

- Combining the cheap storage of lakes with the transactional capabilities and schema enforcement of warehouses.

Data Ingestion Frequency Spectrum

- ❖ Batch
 - Processing large chunks at scheduled intervals (e.g., Daily Tax Reports). Slowest latency.
- ❖ Micro-batch
 - Processing small chunks frequently (e.g., every minute). Common in "Near Real-Time" systems.
- ❖ Real-Time
 - Event-by-event processing. Essential for latency-sensitive actions like fraud detection.



Data Ingestion: Batch Processing

❖ Scheduled Data Movement

- Extracting and loading large, bounded volumes of data at regular, scheduled intervals (e.g., hourly, nightly, weekly).

❖ Characteristics

- High throughput, high latency. It focuses on moving massive amounts of data efficiently rather than quickly.

❖ Typical Use Cases

- Traditional BI reporting, end-of-day financial reconciliation, migrating historical logs, and full database snapshots.

❖ Tools

- Apache Airflow (for scheduling/orchestration).

Data Ingestion: Stream Processing

❖ Continuous, Real-Time Flow

- Processing unbounded data constantly as it is generated, moving records individually or in very small "micro-batches."

❖ Characteristics

- Low latency, variable throughput. The ingestion system is "always on" and reacts instantly to new events.

❖ Typical Use Cases

- Real-time analysis , fraud detection, operational monitoring, and feeding machine learning models.

❖ Tools

- Apache Kafka, Amazon Kinesis, Google Cloud Pub/Sub, Apache Flink.

The Middle Ground: Micro-Batching

❖ Blending the Paradigms

- Processing data in small, discrete chunks at very frequent intervals (e.g., every 1 to 5 minutes).
- Treats a continuous stream as a series of tiny batches.

❖ Benefits of the Hybrid Approach

- Offers near-real-time latency without the extreme engineering complexity of continuous streaming.
- Works exceptionally well with distributed processing frameworks like Apache Spark Structured Streaming.

Advanced Ingestion: Change Data Capture (CDC)

❖ The Problem

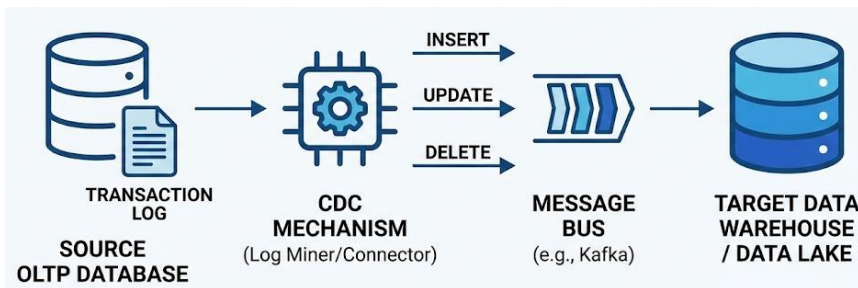
- Querying a live production database (OLTP) to find out "what changed since yesterday" degrades performance and risks downtime for the main application.

❖ The CDC Solution

- Tapping directly into the database's internal transaction log (e.g., PostgreSQL's WAL or MySQL's binlog).
- When an INSERT, UPDATE, or DELETE happens, a specialized tool reads that log and immediately publishes the event.

❖ Why It Is Powerful

- Low performance impact on the source.
- Captures every single state change.



Choosing the Right Strategy: Batch vs. Streaming

❖ Complexity

- Batch is relatively simple to build, debug, and monitor.
- Streaming architectures are notoriously difficult, requiring engineers to handle complex state, late-arriving data, and out-of-order events.

❖ Cost

- Batch is generally cheaper because compute infrastructure spins up, runs the job, and spins down.
- Streaming infrastructure must run 24/7 to listen for events.

❖ The Engineering Rule of Thumb

- "If you don't need real-time, don't build real-time." Default to batch processing unless the business value of millisecond-fresh data clearly outweighs the significant engineering overhead.

An alternative strategy: Data Sharing

❖ Zero-Copy Ingestion

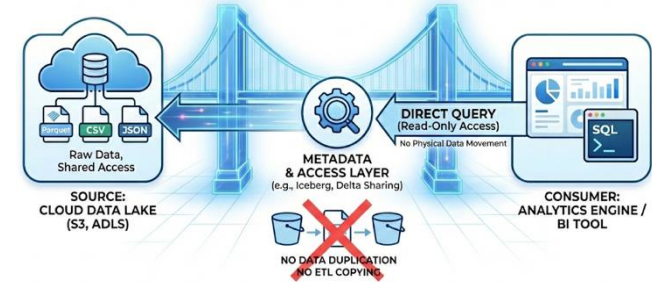
- Consuming and querying external data without physically moving or copying it into your own storage.
 - i.e., one provider grants Read-Only access to their tables.

❖ Benefit

- No ingestion pipeline to build.
- Provides instant access to live, constantly updated data with zero latency.

❖ Constraint

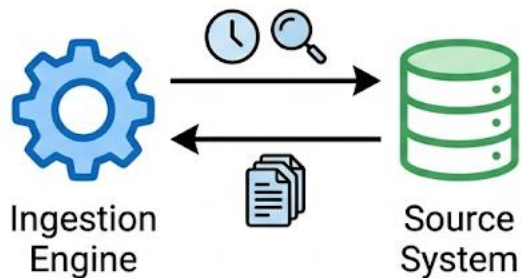
- Lack of physical ownership: If the data provider revokes access or unexpectedly alters the schema, your downstream processes will break instantly.



Synchronous vs. Asynchronous Ingestion

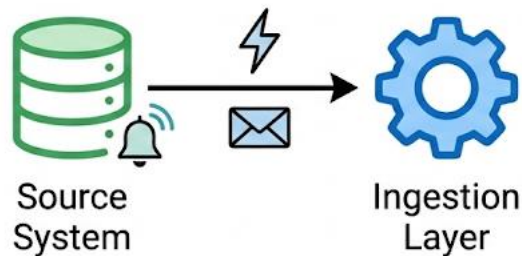
INGESTION STRATEGIES: PULL VS. PUSH

PULL STRATEGY



Periodically requests data.
Best for rate-limiting.

PUSH STRATEGY



Actively sends data as events
occur. Ideal for low-latency.

The Landscape of Data Sources

❖ Diverse Origins Require Diverse Strategies

❖ Web & APIs

- Highly volatile, nested JSON data from SaaS platforms.

❖ IoT & Sensors

- High-frequency streams. Often requires "Edge" ingestion to handle volume before cloud.

❖ Enterprise Systems

- ERP, CRM, and legacy RDBMS. Requires strict transactional integrity and often uses JDBC/ODBC.

❖ Unstructured Media

- Video, Audio, and Images requiring specialized storage (Object Store).

Evolution of File Formats

❖ Practical Issues with Legacy Formats

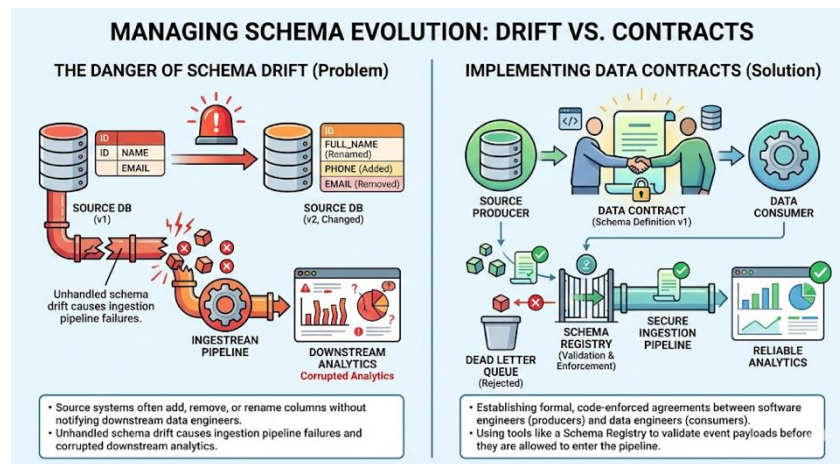
- CSV Issues: No standard schema, no type safety, and delimiter collisions (e.g., commas in text fields).
- JSON: Flexible but verbose. Good for interchange, bad for heavy analytics due to size.

❖ Target Format Optimization

- Converting human-readable formats into optimized columnar formats (Parquet, Iceberg) immediately upon landing.
- Ensuring the chosen target format aligns with the downstream query engine's capabilities and compression needs.

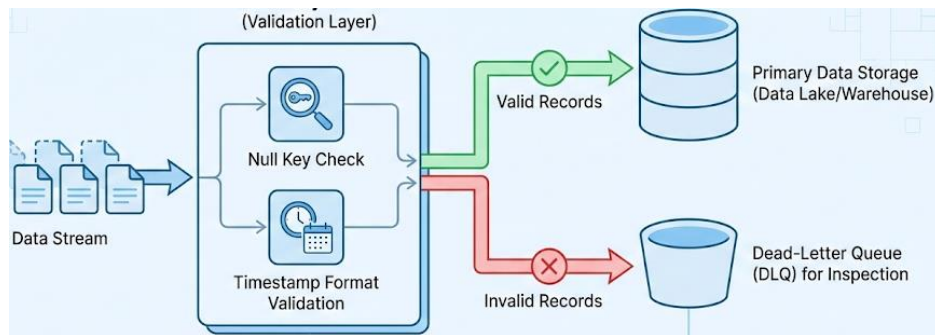
Schema Management and Data Contracts

- ❖ The Danger of Schema Drift
 - Source systems often add, remove, or rename columns without notifying downstream data engineers.
 - Unhandled schema drift causes ingestion pipeline failures and corrupted downstream analytics.
- ❖ Implementing Data Contracts
 - Establishing formal, code-enforced agreements between software engineers (producers) and data engineers (consumers).
 - Using tools like a Schema Registry to validate event payloads before they are allowed to enter the pipeline.



Enforcing Data Quality at the Ingestion Boundary

- ❖ The "Garbage In, Garbage Out" Rule
 - Catching severe data anomalies before they pollute the central storage layer.
 - Protecting downstream consumers from fundamentally broken data.
- ❖ Boundary Checks
 - Implementing lightweight validation, such as checking for null primary keys or validating timestamp formats.
 - Routing bad records to Dead-Letter Queues (DLQs) for inspection without halting the entire pipeline.



In-Flight Processing and Lightweight Transformations

❖ Transforming During Transit

- Applying minor, necessary changes to the data while it is moving, before it lands in storage.
- Keeping these transformations simple to avoid slowing down the ingestion throughput.

❖ Common In-Flight Operations

- Masking or hashing Personally Identifiable Information (PII) to meet security and compliance requirements.
- Flattening heavily nested JSON payloads to simplify downstream SQL queries.

The Location of Transformation: ETL vs. ELT

❖ ETL (Extract-Transform-Load)

- Transform data before storage. Essential for PII masking or when storage is constrained.

❖ ELT (Extract-Load-Transform)

- Load raw data immediately, then transform in the warehouse. Favored in cloud for auditability ("Schema on Read").

❖ Trend

- ELT allows Analysts to own business logic in SQL, decoupling them from Data Engineers.

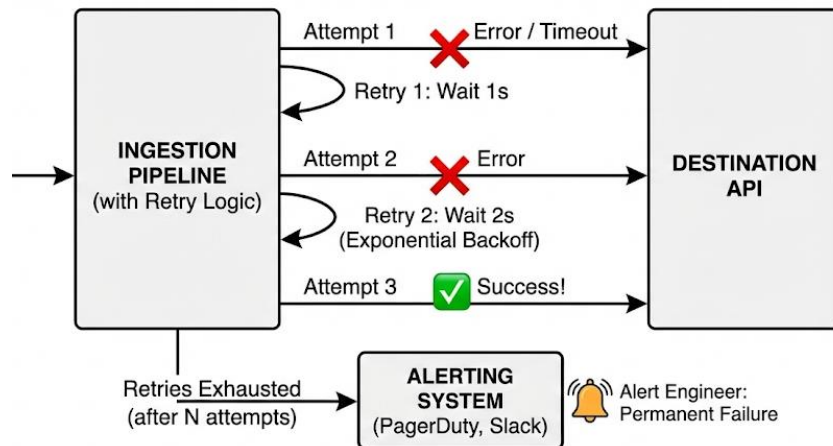
Pipeline Reliability: Handling Failures and Retries

❖ Designing for Failure

- Operating under the assumption that network timeouts, API rate limits, and database locks will occur regularly.
- Decoupling extraction logic from loading logic to isolate failures.

❖ Retry Mechanisms

- Implementing exponential backoff strategies to handle transient API or network errors gracefully.
- Alerting engineers only when automated retries have completely exhausted their limits.



Building Idempotent Ingestion Pipelines

❖ The Concept of Idempotency

- An operation that can be run multiple times without changing the result beyond the initial application.
- "Running this ingestion job twice should never duplicate the data."

❖ Implementation Techniques

- Using upserts (Merge) based on unique primary keys instead of blind append operations.
- Overwriting specific date partitions completely if a daily batch job needs to be re-run due to an error.

Orchestration and Job Scheduling

❖ Managing the Pipeline Flow

- Moving beyond simple cron jobs to sophisticated, dependency-aware orchestrators.
- Ensuring ingestion job B only starts when extraction job A successfully finishes.

❖ Industry Standard Tools

- Using platforms like Apache Airflow to define pipelines as code.
- Managing complex schedules, backfills, and cross-system dependencies from a central control plane.

Security, Privacy, and Data Governance

❖ Securing Data in Transit

- Enforcing TLS/SSL encryption for all data moving over the network from source to destination.
- Managing API keys and database credentials via secure secret stores (e.g., AWS Secrets Manager).

❖ Access Control

- Ensuring the ingestion service account operates strictly with the principle of least privilege.
- Logging all ingestion activities to maintain a clear audit trail for compliance.

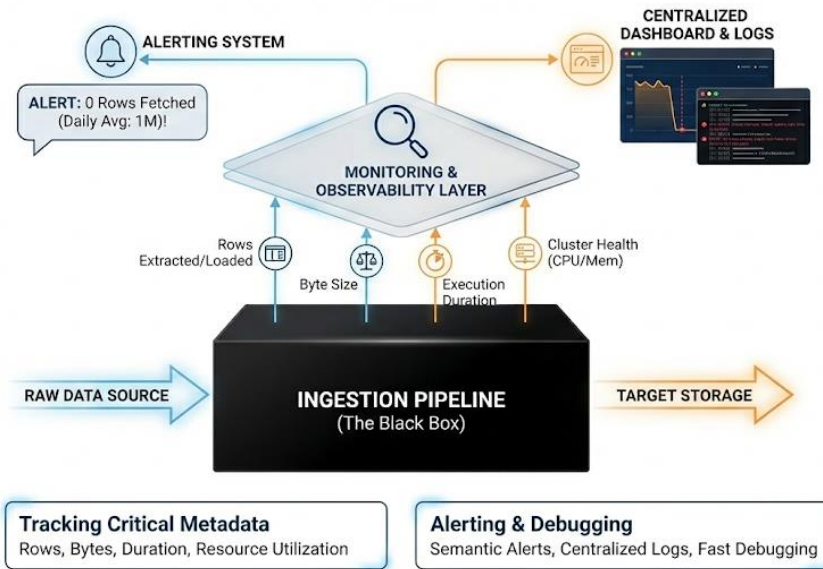
Monitoring, Logging, and Pipeline Observability

❖ Visibility into the "Black Box"

- Tracking critical metadata: rows extracted, rows loaded, byte size, and execution duration.
- Monitoring the health and resource utilization of the ingestion compute clusters.

❖ Alerting and Dashboards

- Setting up semantic alerts (e.g., triggering an alert if extracting 0 rows when the daily average is 1 million).
- Centralizing logs to quickly debug pipeline failures without manually accessing individual servers.



Summary: Modern Ingestion Principles

- ❖ Decouple
 - Use Asynchronous patterns and ELT to reduce fragility.
- ❖ Automate
 - Use Managed Connectors and Orchestrators to remove manual toil.
- ❖ Resilience
 - Design for failure (DLQs, Idempotency).
- ❖ Context
 - Understand your data's frequency, shape, and volume before writing code.

Further Reading & Resources

- ❖ Reis & Housley, Fundamentals of Data Engineering,
 - Chapter 7
 - <https://learning.oreilly.com/library/view/fundamentals-of-data/9781098108298/>

